

COMP90042 NLP

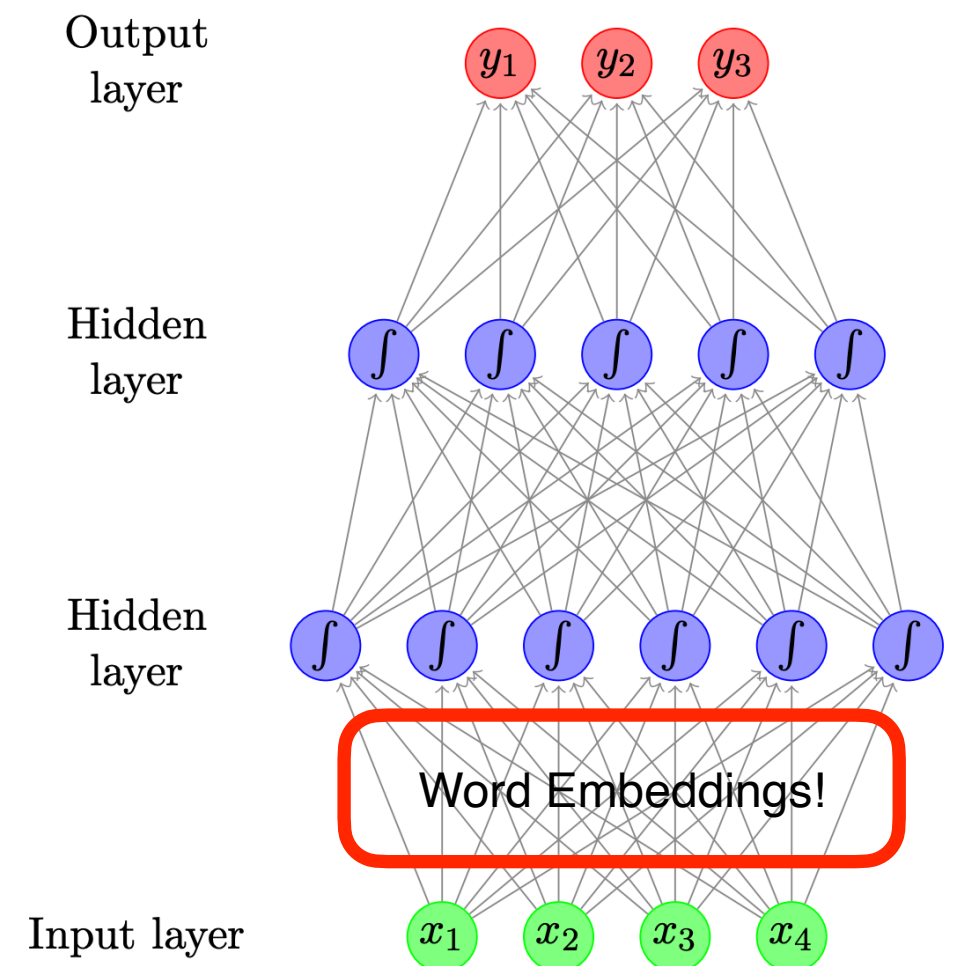
L11: Pretrained Models



Jey Han Lau

Pretrained Word Vectors

- Pretrained word vectors are useful
- For NLP models, the first layer can be initialised with pretrained word embeddings instead of initialised with random weights
- Easy performance gain across a range of NLP tasks



Pretrained Models

- Even with pretrained word embeddings, most of the network weights are still randomly initialised
- Can we go one step further? Why not use a **whole pretrained network** for downstream tasks?

Outline

- GPT
- BERT
- T5



GPT

GPT-1 (2018)

- https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- A transformer-based language model
- Pretraining objective: next word prediction

Training/Model Details

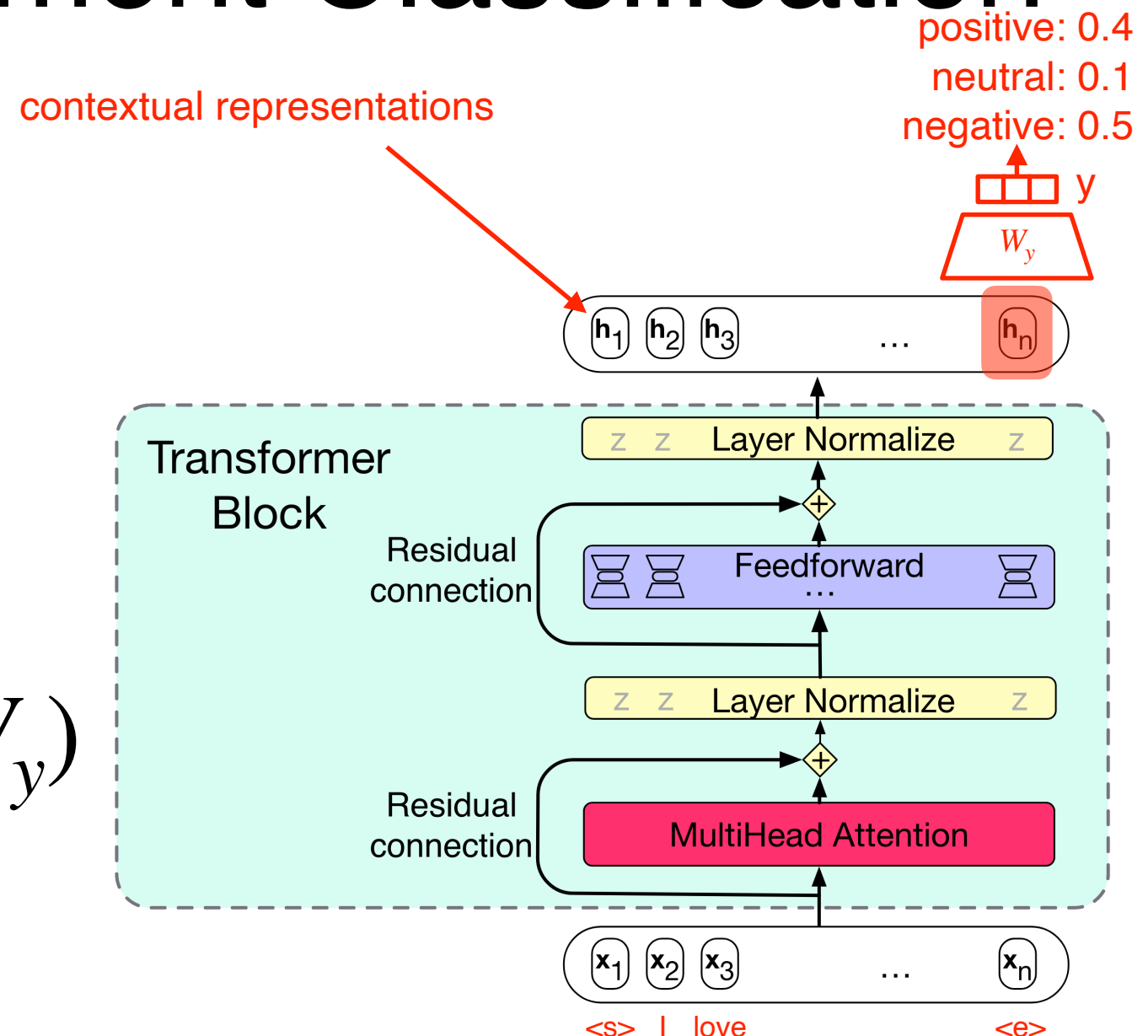
- Subword tokenisation: byte-pair encoding
- 12 layers of transformer (110M parameters)
- Pretraining corpus: BookCorpus
 - 7K unique books of varying genres; long stretches of contiguous text

Adapt To Downstream Tasks

- Given pretrained GPT, how do we use it for downstream tasks?
- For classification tasks, add an additional classification layer after processing the whole input
 - Classification layer is randomly initialised
- Continue training **the whole network** using the downstream task dataset
 - AKA 'fine-tuning'

Fine-tuning: Sentiment Classification

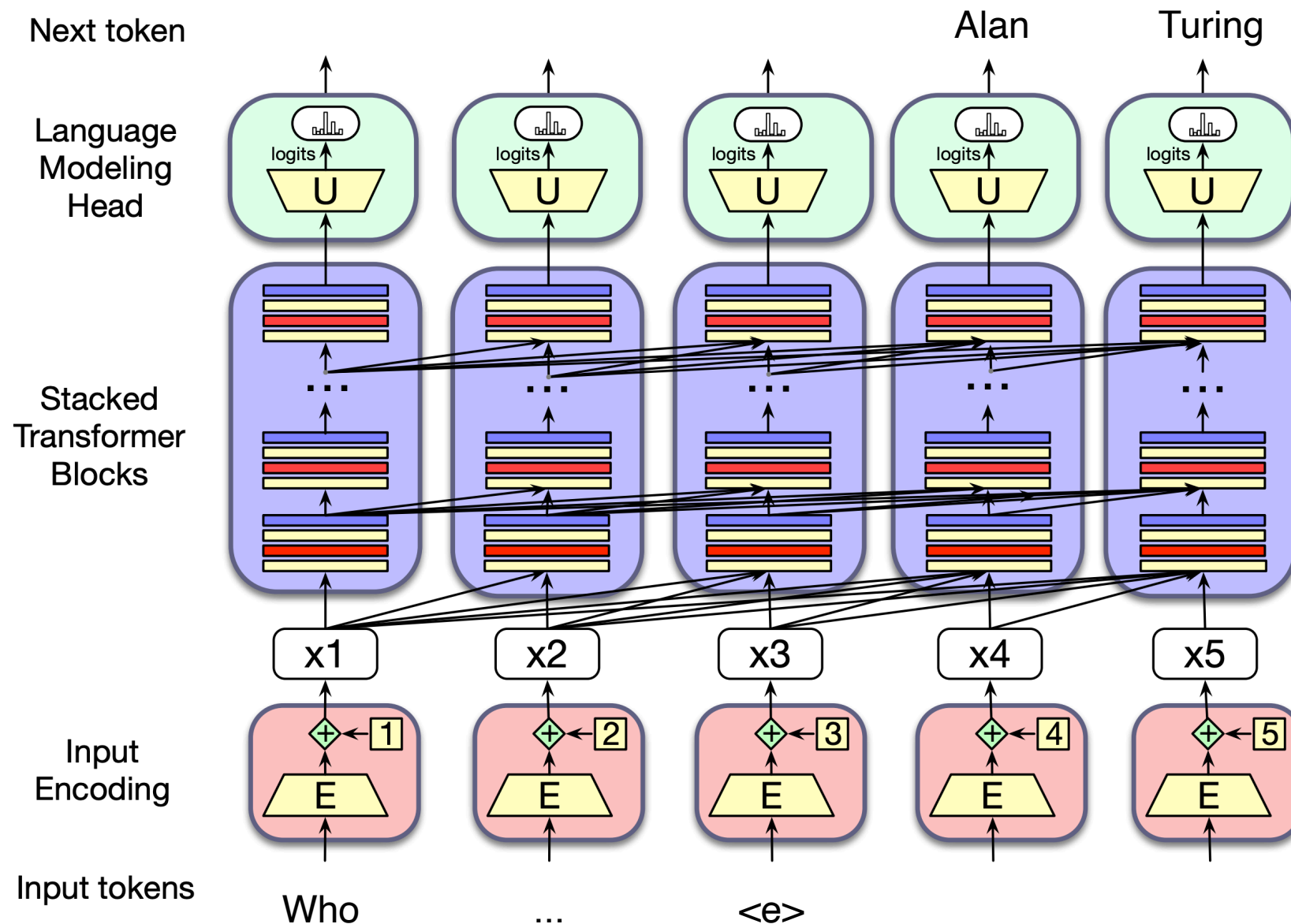
- Initialise a classification layer (W_y), and use last word's output (h_n) as input
- $P(y | x) = \text{softmax}(h_n W_y)$
- Training Instance:
 $\langle s \rangle$ *I love pizza!* $\langle e \rangle$
 = positive
- Loss = $\log(0.4)$



Fine-tuning: Question Answering

$$\text{Loss} = \text{logprob}(\text{Alan}) + \text{logprob}(\text{Turing})$$

$\text{logprob}(\text{Alan})$ $\text{logprob}(\text{Turing})$



Training instance:
 Question: Who invented the Turing Machine?
 Answer: Alan Turing

How Good is GPT?

- Works much better!

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.9	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	88.1	91.3	45.4	80.0	82.3	56.0	75.2

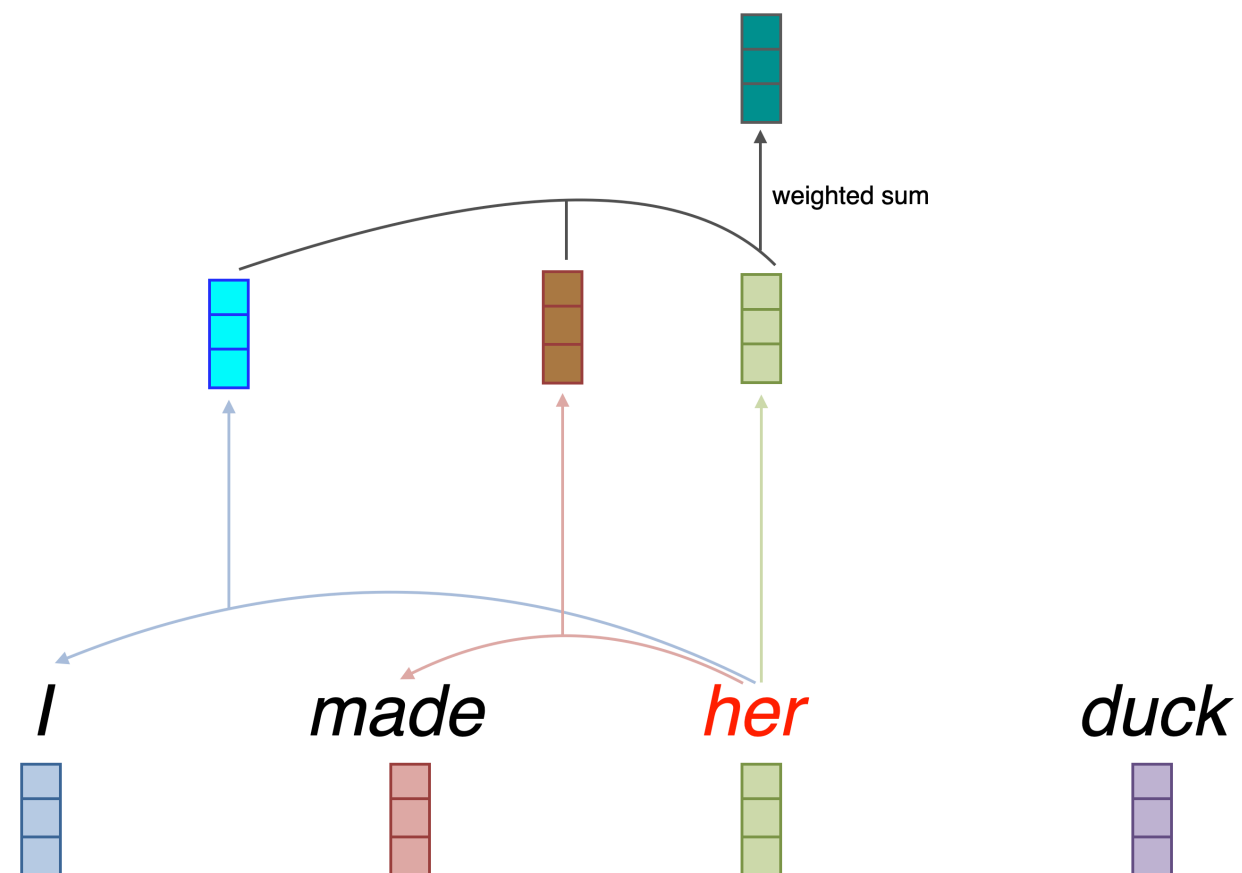
- MNLI, RTE: textual entailment
- QQP, STS-B, MRPC: sentence similarity
- QNLI: answerability prediction
- SST-2: sentiment analysis
- COLA: sentence acceptability prediction



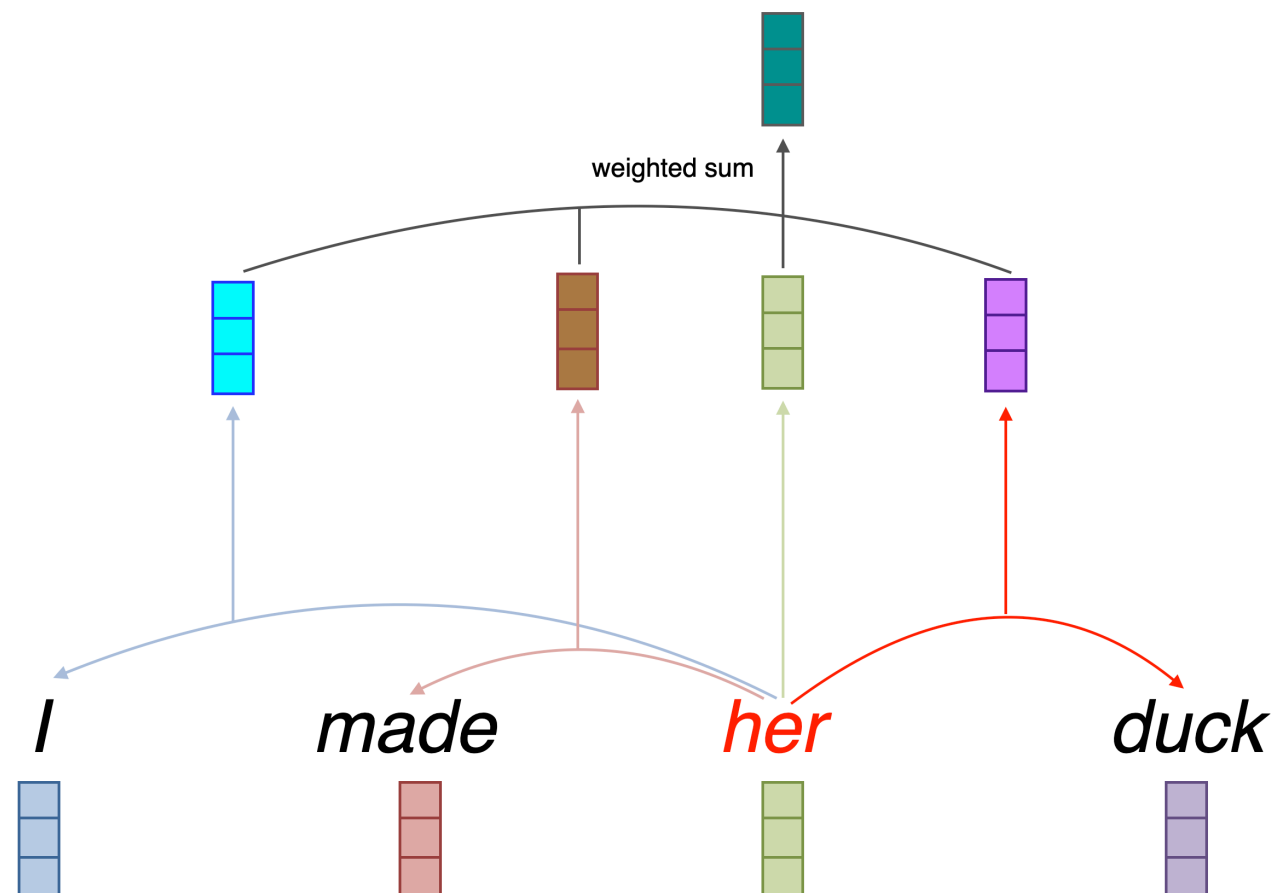
BERT

BERT: **B**idirectional **E**ncoder Representations from **T**ransformers (2018)

- <https://arxiv.org/abs/1810.04805>
- Uses **bidirectional** self-attention instead
- That is, each target word attends to **both** left and right context words to build its contextual representation



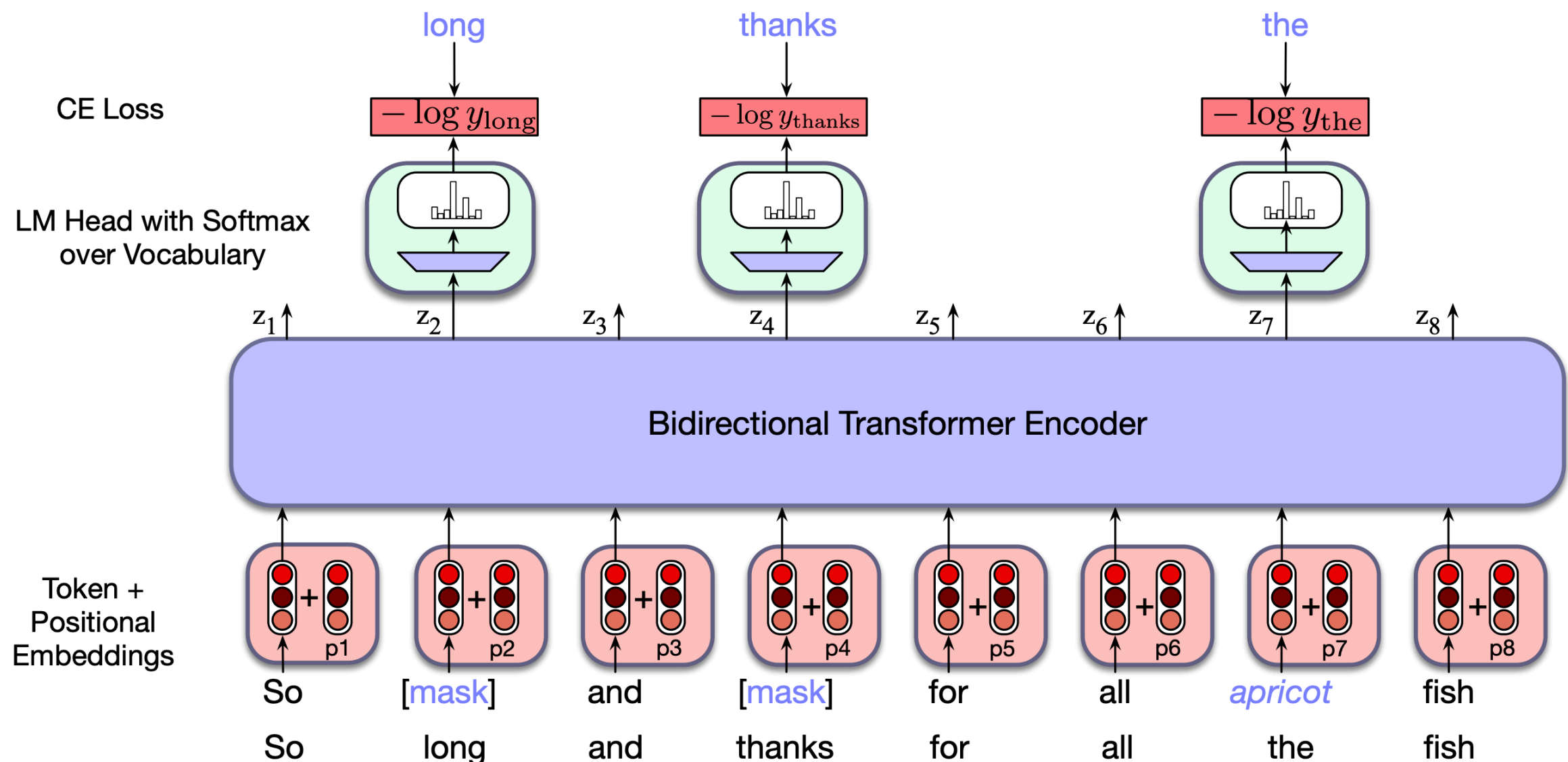
GPT =
Unidirectional or
Causal Attention



BERT =
Bidirectional or
Full Attention

Objective: Masked Word Prediction

- ‘Mask’ out k% of tokens at random
- Objective: predict the masked words



Training/Model Details

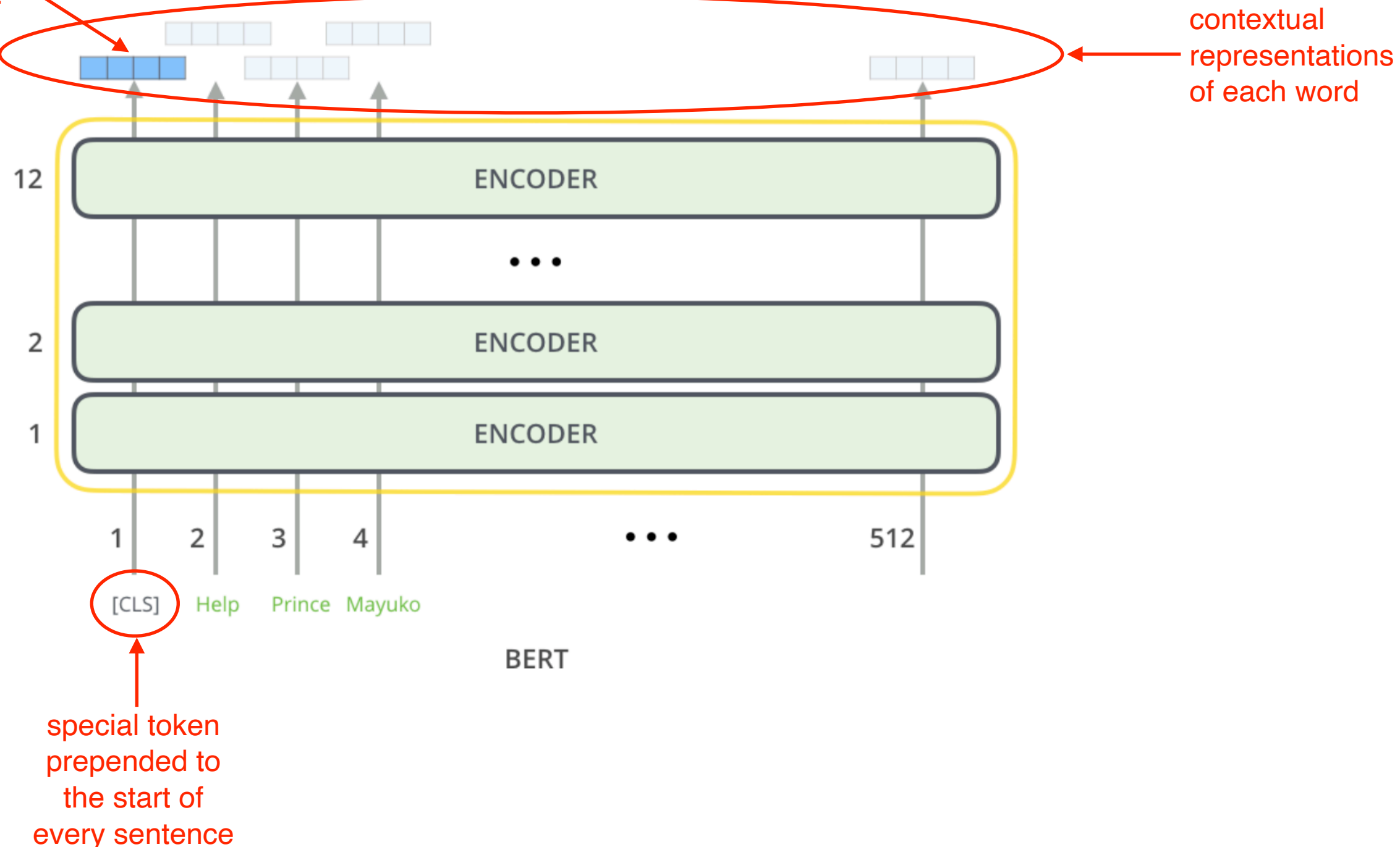
- Subword tokenisation: WordPiece
- BERT-base: 12 layers of transformers (110M)
- BERT-large: 24 layers of transformers (340M)
- BERT is pretrained on Wikipedia+BookCorpus
- Training takes multiple GPUs over several days

Adapt To Downstream Tasks

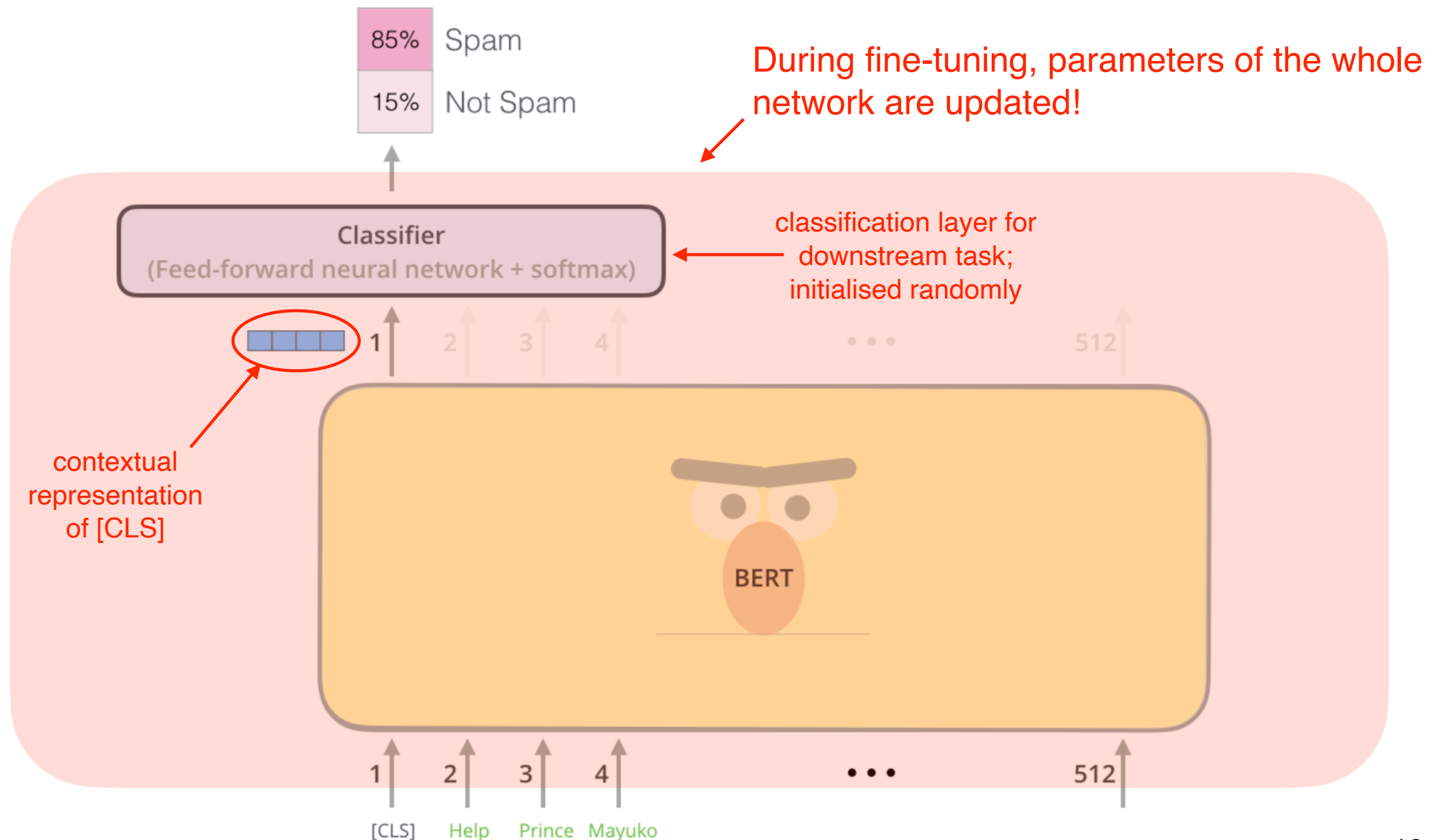
- Given pretrained BERT, how do we use it for downstream tasks?
- Same as GPT - add a classification layer
 - But in this case, we use a special first token ([CLS]) as the input to the classification layer

Example: Spam Detection

captures information
of the whole sentence;
input to new
classification layer
we're adding



Example: Spam Detection



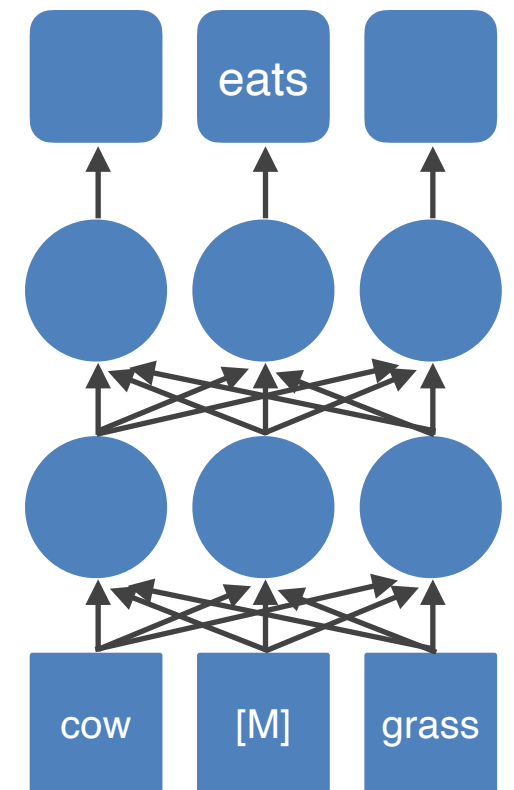
How Good is BERT?

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.9	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	88.1	91.3	45.4	80.0	82.3	56.0	75.2
BERT _{BASE}	84.6/83.4	71.2	90.1	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	91.1	94.9	60.5	86.5	89.3	70.1	81.9

- MNLI, RTE: textual entailment
- QQP, STS-B, MRPC: sentence similarity
- QNLI: answerability prediction
- SST-2: sentiment analysis
- COLA: sentence acceptability prediction

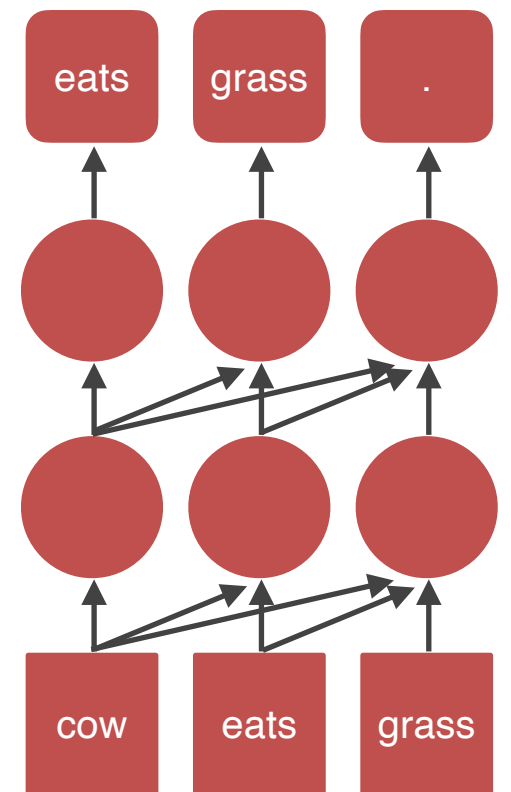
Encoder

- BERT's bidirectional transformer is also called a **transformer encoder**
- Each output word is conditioned on bidirectional context (bidirectional attention)
- Better contextual representation
- Unable to generate text post-training
 - Doesn't work for generation tasks, e.g. machine translation



Decoder

- GPT's unidirectional transformer is also called a **transformer decoder**
- Each output word is conditioned on only left context words (unidirectional attention)
- Works on both classification and generation tasks
- But performance on classification task not as good as encoder models



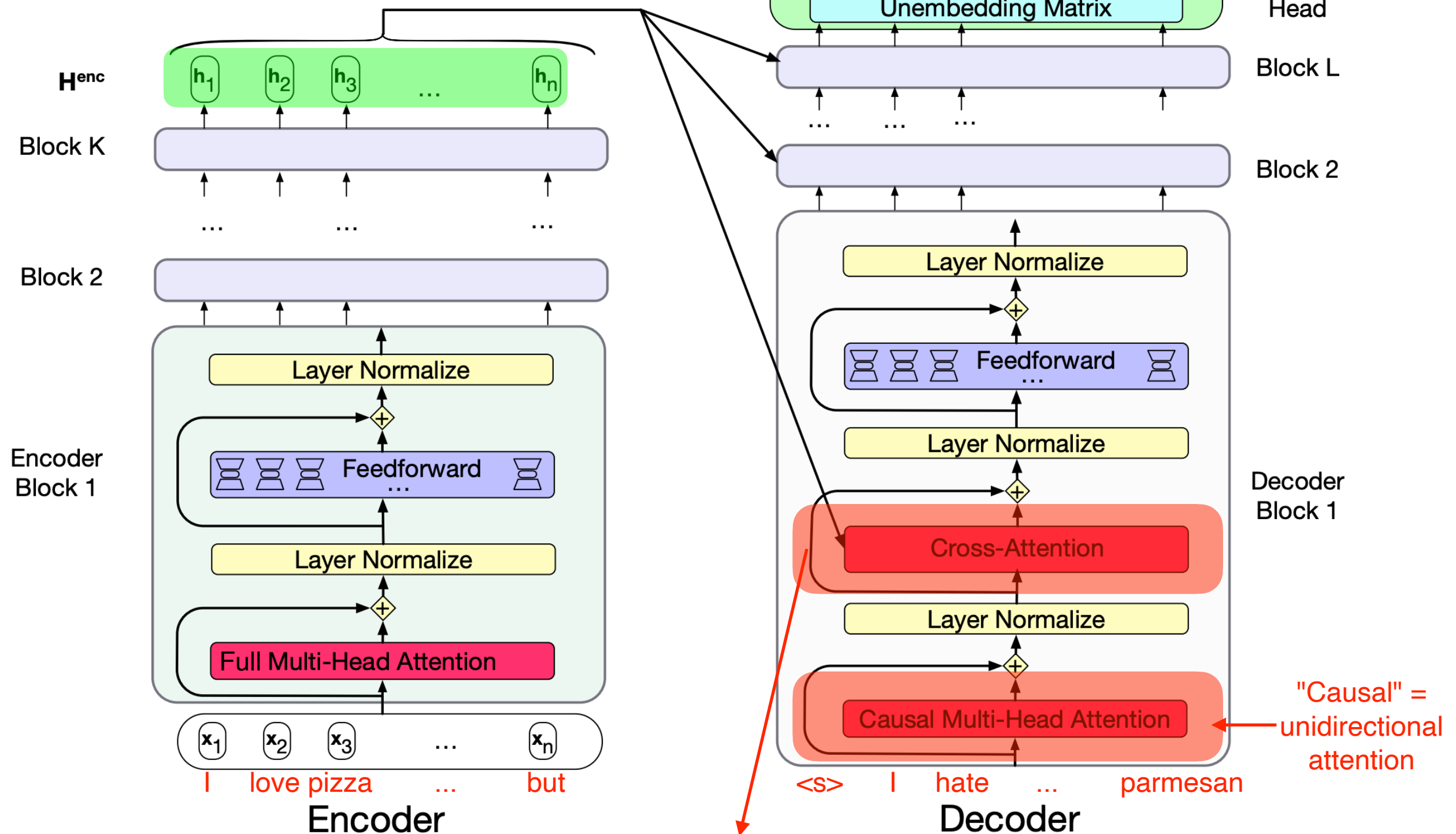
Encoder-decoder

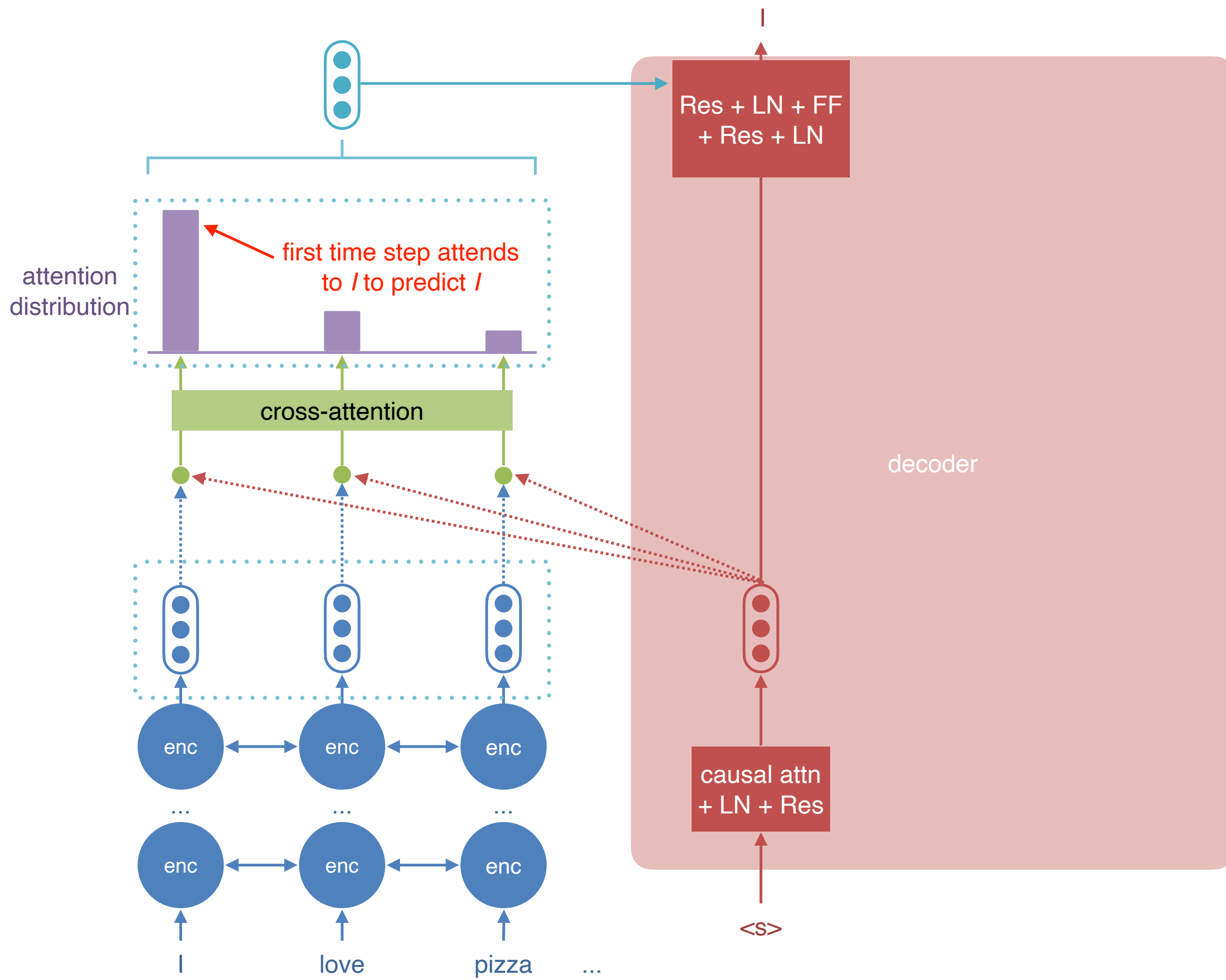
- How can we marry the benefits of encoder (strong classification performance but can't do generation) and decoder (poor classification performance but can do generation)?
- Combine them!
- Use an encoder to process an input
- Use a decoder to generate the output

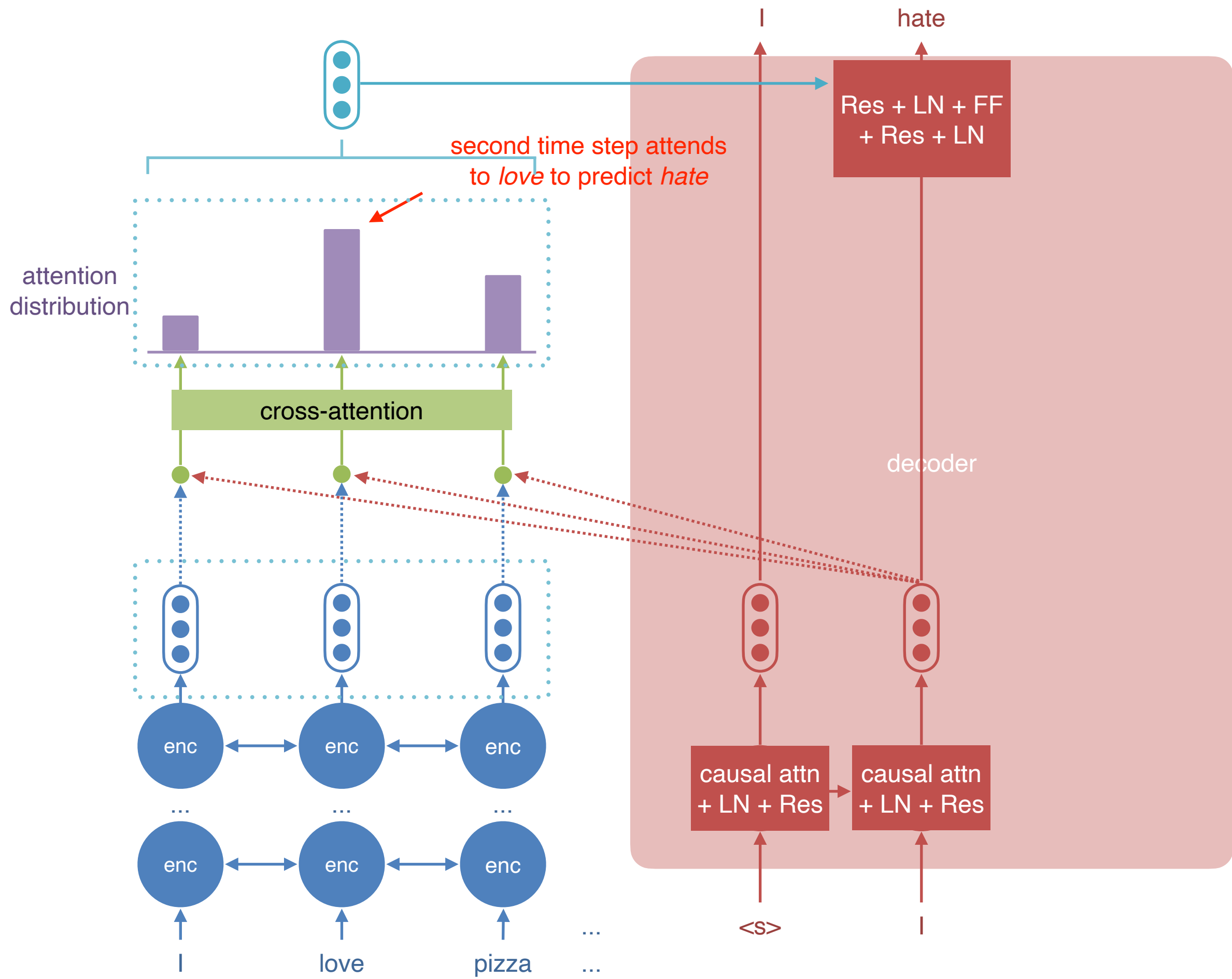
Sentence completion:

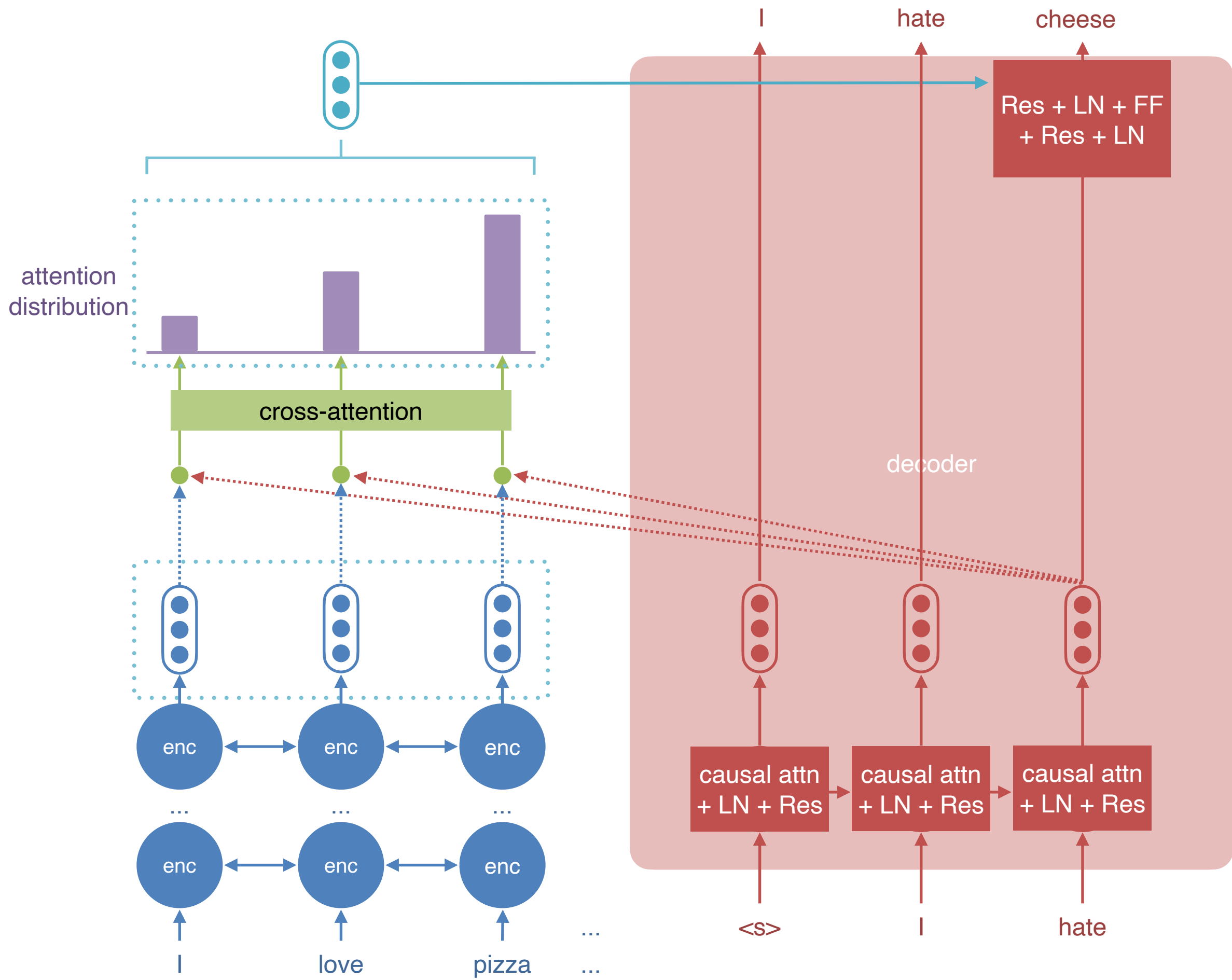
I love pizza and burger but

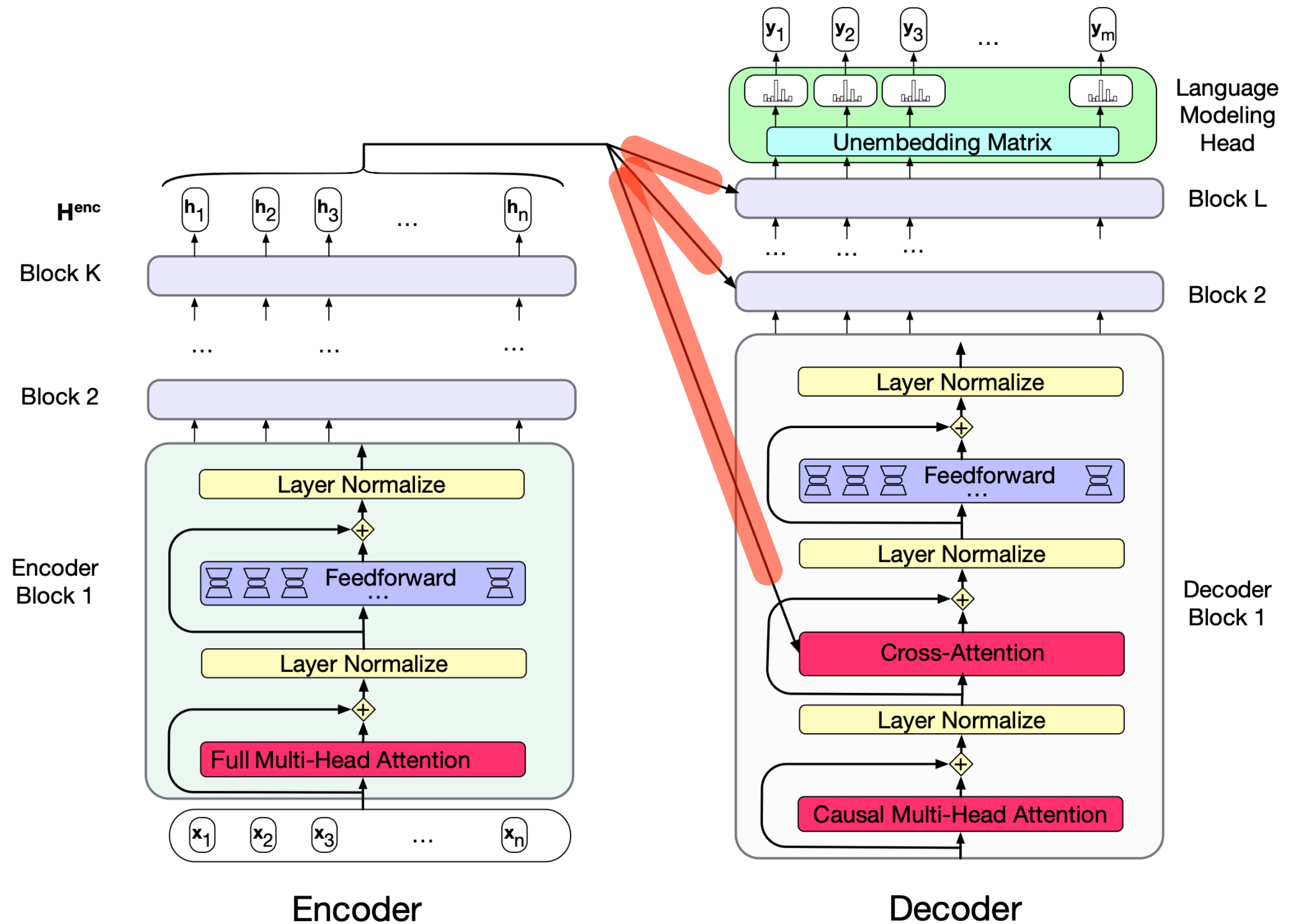
I hate cheese although I am OK with parmesan.









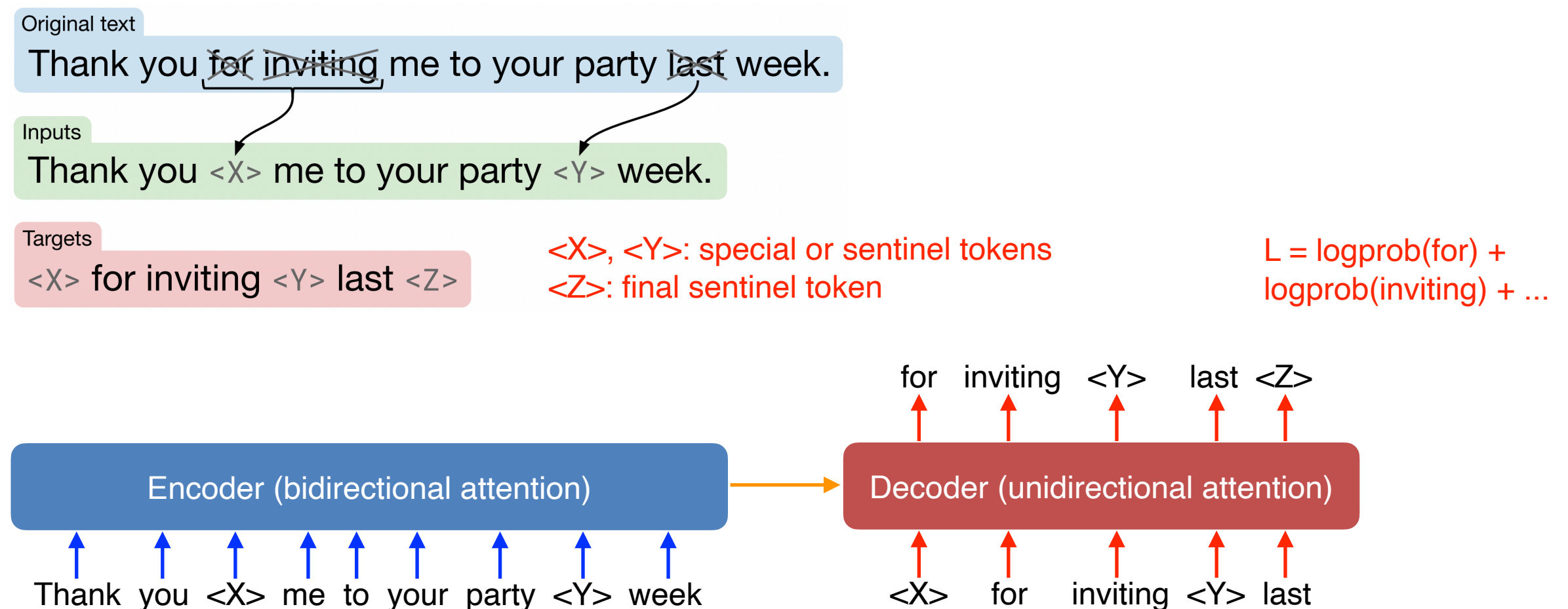


- Cross-attention happens at every decoder layer
- Always uses the output from the **last encoder layer**

T5

T5 (2019)

- Text-to-text Transfer Transformer: <https://jmlr.org/papers/volume21/20-074/20-074.pdf>
- Pretraining objective: span corruption



Other Alternative Objectives

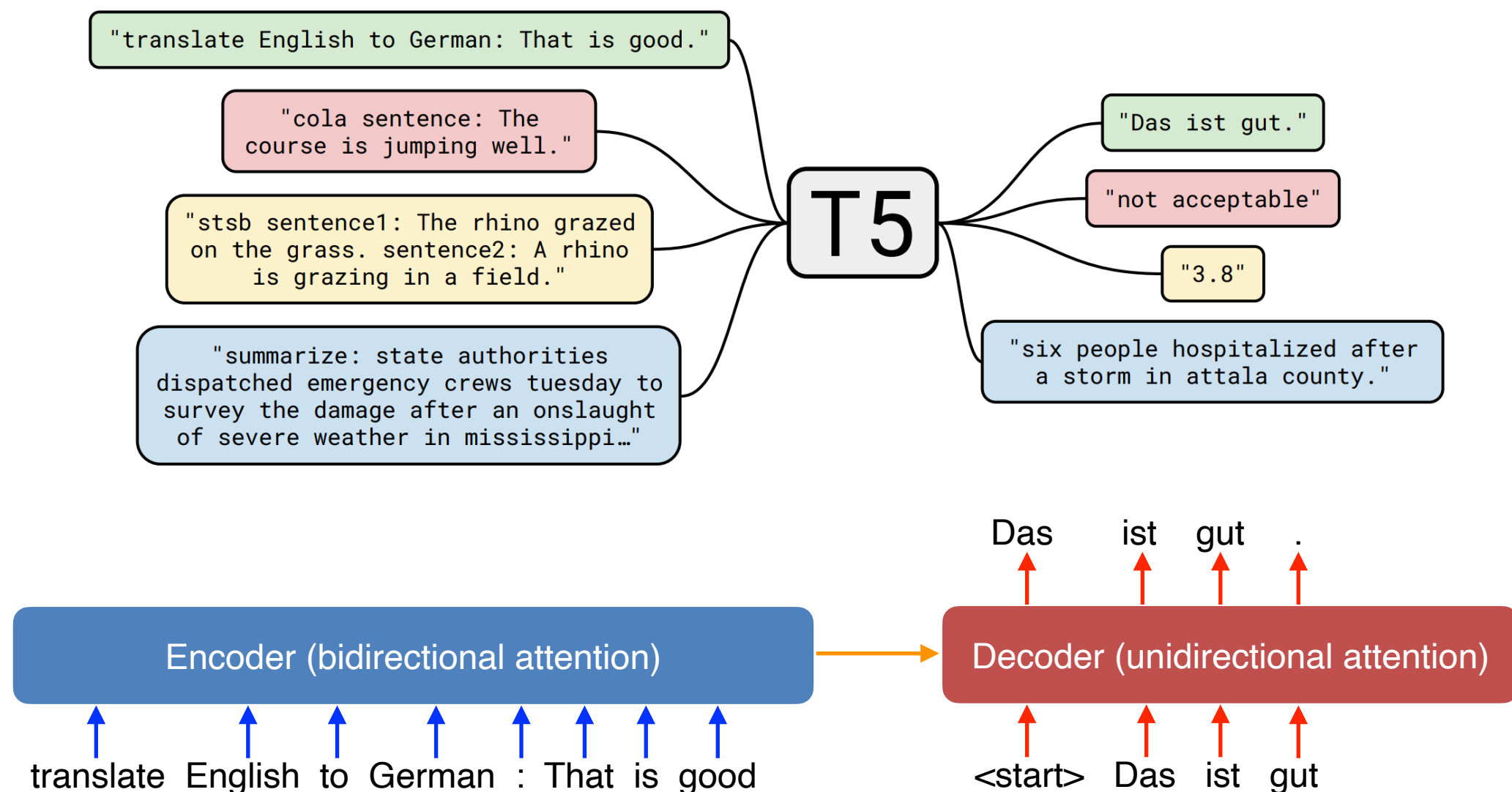
- Prefix language model:
 - *cows love to → eat grass*
 - Masked language model:
 - *cows <M> <M> eat grass → cows love to eat grass*
 - Shuffling:
 - *grass cows to love eat → cows love to eat grass*
 - Drop tokens:
 - *cows love grass → to eat*
- But span corruption was found to work best empirically

Pretraining Corpus

- Filtered C4: filtered web text
 - Lots of noise: non-language, non-English and offensive content
 - Heuristics applied to filter them
- Quality matters (not just quantity)

Fine-tuning

- Cast all tasks as a text-to-text problem, and fine-tune with next word prediction objective



How Good is T5?

- Comes with a few sizes: small (60M), base (220M), large (770M), 3B and 11B.

Model	GLUE Average	CNN/DM ROUGE-L	WMT EnFr BLEU
Previous best	89.4 ^a	40.63 ^g	43.8^e
T5-Small	77.4	38.35	36.0
T5-Base	82.7	39.40	41.2
T5-Large	86.4	39.75	41.5
T5-3B	88.5	39.94	42.6
T5-11B	90.3	40.69	43.4

- GLUE: large benchmark with over 9 language understanding tasks
- CNN/DM: summarisation
- WMT: machine translation
- Performance of largest model close to human

A Final Word

- Pretrain-fine-tune = new paradigm for building NLP models
- When we use them for downstream tasks, we are no longer starting from “scratch”
- Scaling appears to continue to improve performance
- Larger and larger models are developed → large language models (next lecture!)

Readings

- BERT: <https://arxiv.org/abs/1810.04805>
- GPT-1: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- T5: <https://jmlr.org/papers/volume21/20-074/20-074.pdf>